

Name of the Student	SHAM S.
Reg. No	192325076
GitHub Link	https://github.com/sham-2005/MLA0403-deep_learning

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning
SLOT :

COURSE CODE: MLA04
Faculty : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Course Outcome Weightage: 35%

Duration: 90 minutes

Assessment Tool Weightage: 30%

Mark : 25

Code of Conduct:

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Name of the student:

Criteria	Subject Knowledge and Conceptual Understanding (2)	Technical Accuracy and Application of Concepts (3)	ProblemSolving and Analytical Thinking (2)	Communication Skills and Clarity of Explanation (2)	Confidence, Presentation and Professional Behaviour (1)	Total (10)
Weightage	20 %	30%	20%	20 %	10 %	100 %
Q1						
Q2						
Q3						
Q4						

Q5						
Total						

Faculty Signature :

Total Marks:

QUESTIONS: 25

Total Marks:25

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.

- Mention suitable evaluation measures and expected outcomes. □ Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Q1. Design an Autoencoder-Based Feature Learning System

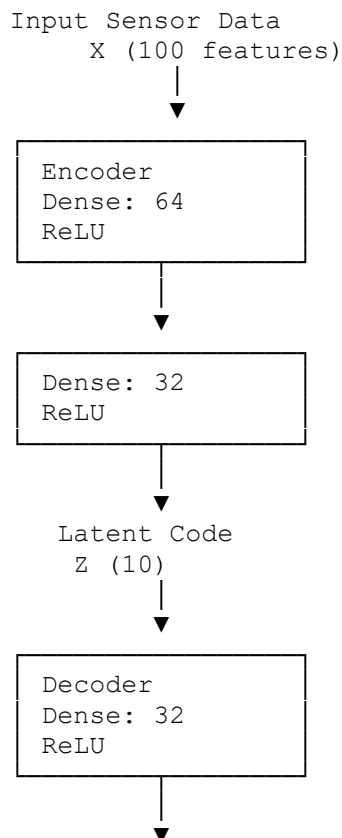
Problem

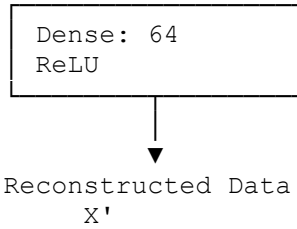
A manufacturing company collects **high-dimensional sensor data** from machines to detect defective products. The large number of sensor features makes analysis difficult. An **Undercomplete Autoencoder (UAE)** can learn a compact representation of the sensor data and reconstruct the original input.

Requirements

- **Input:** High-dimensional sensor features
- **Output:** Reconstructed sensor data and compact features
- **Constraint:** Latent dimension must be smaller than input dimension
- **Objective:** Dimensionality reduction and useful feature extraction

Proposed Architecture





Encoder

The encoder converts the high-dimensional input into a compact latent representation:

$$Z = f_{\theta}(X)$$

For example:

- Input = 100 sensor features
- Hidden layer = 64 neurons
- Hidden layer = 32 neurons
- Latent layer = 10 neurons

ReLU activation is used in the hidden layers.

Decoder

The decoder reconstructs the original sensor input:

$$X' = g_{\phi}(Z)$$

Architecture:

10 → 32 → 64 → 100

The final activation can be selected according to the sensor-data range.

Reconstruction Loss

Mean Squared Error (MSE):

$$L = \frac{1}{n} \sum_{i=1}^n (x_i - x'_i)^2$$

The network minimizes the difference between the original and reconstructed sensor data.

How Dimensionality Reduction Works

The latent layer contains only **10 features compared with the original 100 features**. Therefore, the network is forced to learn the most important information instead of simply copying the input.

The learned latent representation can then be used for:

- Defect classification
- Visualization
- Clustering
- Anomaly detection
- Machine-condition analysis

Training

1. Feed sensor data to the encoder.
2. Generate latent representation.
3. Decode the representation.
4. Compare reconstructed data with original data.
5. Calculate reconstruction loss.
6. Backpropagate the error.
7. Update network weights.

Evaluation

Suitable measures:

- Reconstruction MSE
- MAE
- Classification accuracy using learned features
- Precision, recall and F1-score for defect detection

Expected Outcome

The proposed undercomplete autoencoder reduces the dimensionality of sensor data while preserving important information. The compact latent representation provides useful features for detecting defective products.

Q2. Design a Robust Regularized Autoencoder

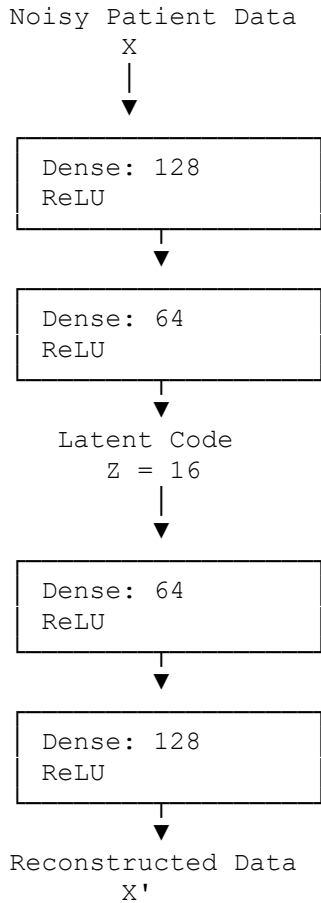
Problem

A healthcare organization has **noisy patient-record features**. Directly training a model on noisy data can cause overfitting and poor feature extraction. A **Regularized Autoencoder** can learn a more robust representation.

Requirements

- **Input:** Noisy patient-record features
- **Output:** Reconstructed patient features
- **Objective:** Learn robust features
- **Constraint:** Avoid overfitting

Proposed Architecture



Regularization Strategy

Use **L2 regularization** on the encoder and decoder weights.

The total objective becomes:

$$L = L_{reconstruction} + \lambda \sum W^2$$

where:

- $L_{reconstruction}$ = reconstruction error
- λ = regularization strength
- W = model weights

Dropout can also be used in hidden layers to reduce dependence on individual neurons.

Training Process

```
graph TD; A[Patient Data] --> B[Add/retain Noise]; B --> C[Encoder]; C --> D[Regularized Latent Representation]; D --> E[Decoder]; E --> F[Reconstructed Data]; F --> G[Reconstruction + Regularization Loss]; G --> H[Backpropagation];
```

Why Regularization is Needed

Without regularization, the autoencoder may memorize the training data. L2 regularization penalizes excessively large weights and encourages a simpler model.

Dropout further improves robustness by randomly disabling neurons during training.

Benefits

- Reduces overfitting
- Learns smoother representations
- Handles noisy features better
- Improves downstream classification
- Extracts useful patient-record features

Evaluation

- Reconstruction MSE/MAE
- Classification accuracy
- Precision
- Recall
- F1-score
- Performance on unseen patient records

Expected Outcome

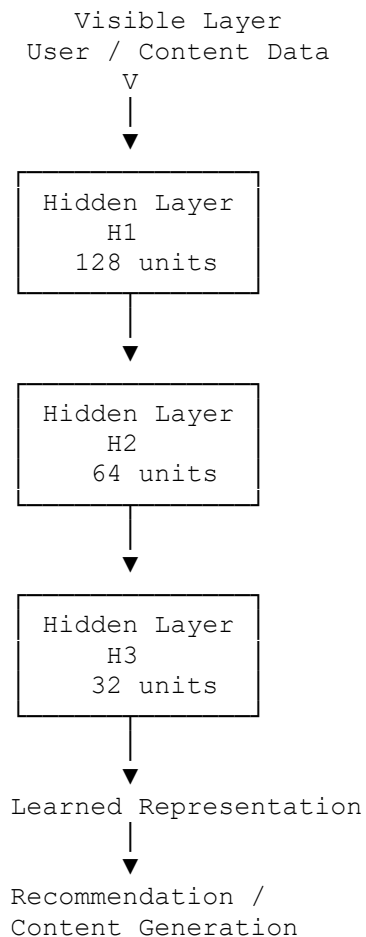
The regularized autoencoder learns a robust latent representation that removes unnecessary noise and improves the quality of features used for downstream healthcare classification.

Q3. Design a Deep Generative Model

Problem

A recommendation/content-generation system needs to learn a **complex probability distribution** of user preferences or content features. A **Deep Belief Network (DBN)** can learn hierarchical representations.

Proposed Model: Deep Belief Network



Architecture

Example:

- Visible layer: 100 input features
- Hidden layer H1: 128 units
- Hidden layer H2: 64 units
- Hidden layer H3: 32 units

The layers learn progressively higher-level representations.

Learning Mechanism

A DBN is constructed using stacked probabilistic models, commonly Restricted Boltzmann Machines (RBMs).

Training can be performed layer by layer:

1. Train the first RBM using visible input.
2. Use its hidden representation as input to the next layer.
3. Train the next RBM.
4. Continue for deeper layers.
5. Fine-tune the complete network for the target application.

Why DBN?

A DBN is suitable because it can learn **hierarchical and complex patterns** from data.

For recommendation:

- Lower layers learn basic user/content relationships.
- Higher layers learn more complex preference patterns.
- The final representation can be used for recommendation or generation.

Evaluation

For recommendation:

- Precision@K
- Recall@K
- F1-score
- NDCG

For generated content:

- Reconstruction/generation quality
- Distribution similarity
- User acceptance or relevance

Expected Outcome

The DBN learns hierarchical representations of complex user or content data and uses these representations to generate relevant recommendations or content.

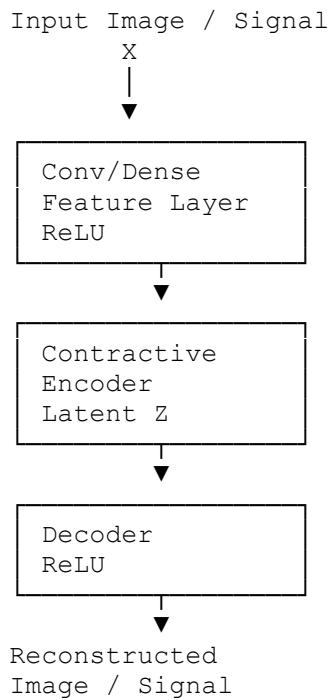
Q4. Design a Stochastic and Contractive Representation Model

Problem

An image or signal-processing system should produce representations that remain stable when the input changes slightly.

A **Contractive Autoencoder (CAE)** can be used because it encourages the learned representation to be insensitive to small input variations.

Architecture



Contractive Objective

The objective combines reconstruction loss with a penalty on the sensitivity of the encoder:

$$L = L_{reconstruction} + \lambda \left\| \frac{\partial h}{\partial x} \right\|_F^2$$

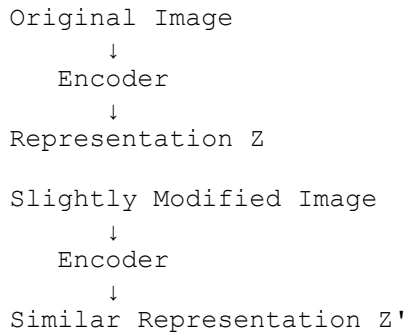
where:

- h = encoder representation
- x = input
- λ = contractive regularization coefficient
- $\|\cdot\|_F^2$ = squared Frobenius norm

Robustness Mechanism

The Jacobian penalty forces the representation to change less when the input changes slightly.

For example:



Therefore, small variations in the input have limited effect on the learned features.

Applications

- Image recognition
- Signal processing
- Noise-resistant feature extraction
- Pattern recognition

Training

1. Input image/signal is passed through encoder.
2. Latent representation is generated.
3. Decoder reconstructs input.
4. Reconstruction error is calculated.
5. Contractive penalty is calculated.
6. Both losses are combined.
7. Backpropagation updates the network.

Evaluation

- Reconstruction error
- Representation stability
- Classification accuracy
- Robustness under small input perturbations

Expected Outcome

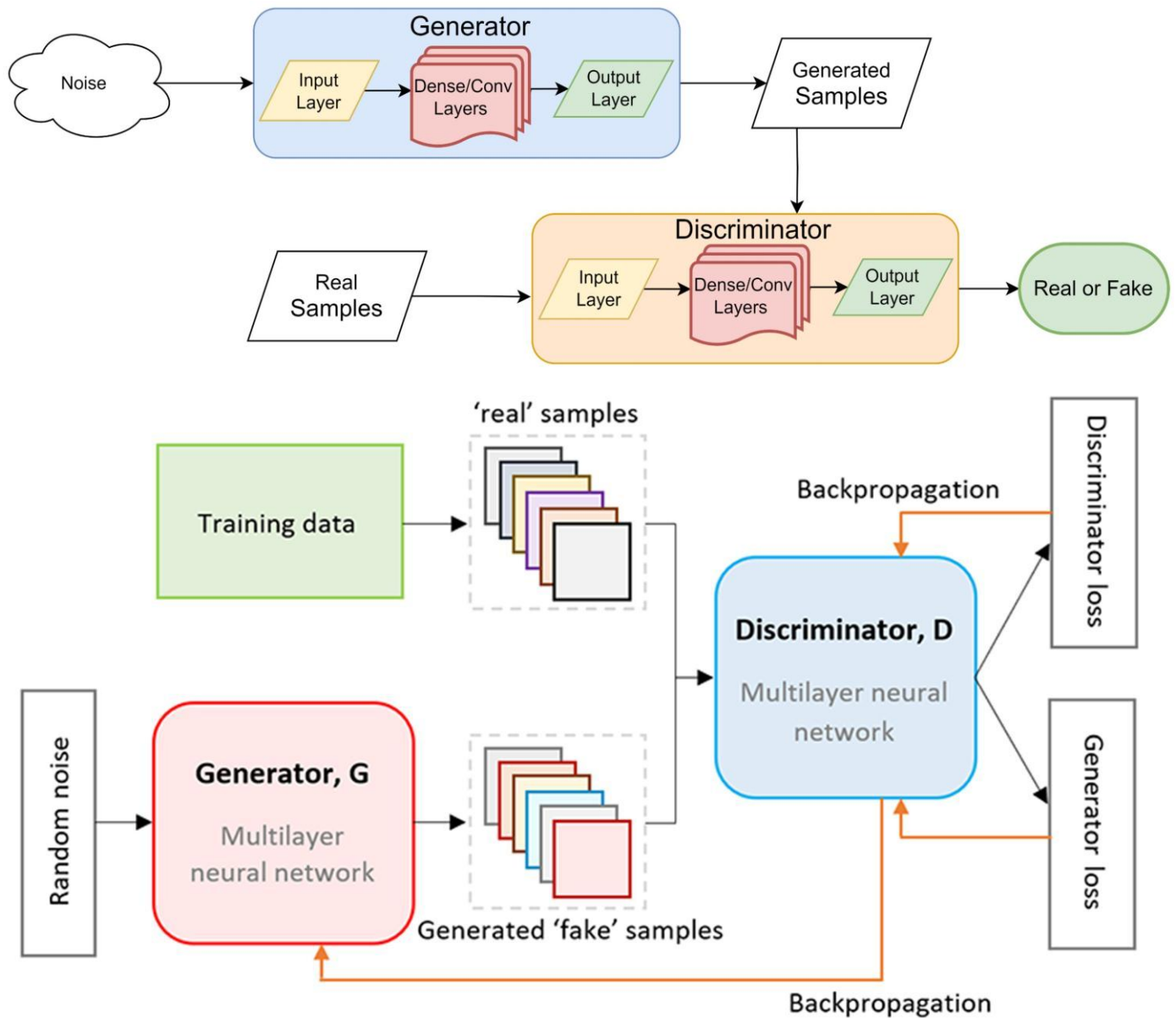
The model learns stable representations that preserve important information while becoming less sensitive to small changes in image or signal inputs.

Q5. Design a GAN-Based Data Generation System

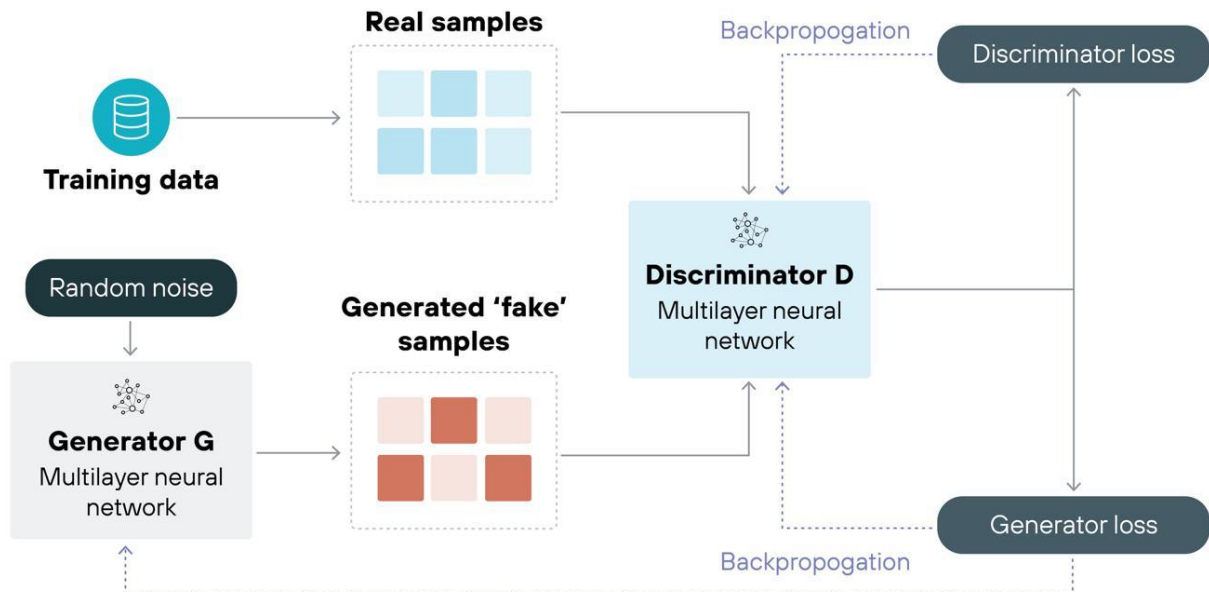
Problem

An organization has a limited number of training images. A **Generative Adversarial Network (GAN)** can generate synthetic images that resemble the training data.

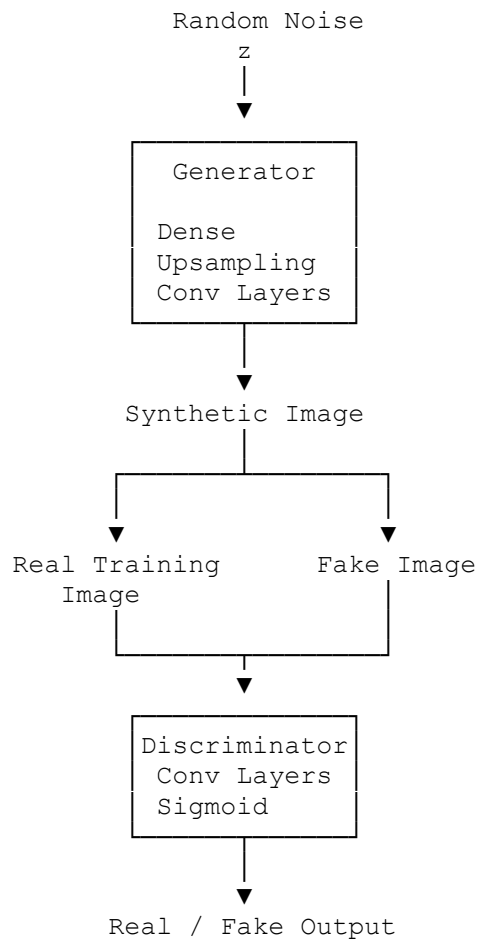
GAN Architecture



Generative adversarial network (GAN) architecture



5



Generator

The generator takes a random latent vector:

$$z \sim p(z)$$

and converts it into a synthetic image:

$$G(z) = X_{fake}$$

Example architecture:

```
Noise vector: 100
  ↓
Dense: 256
  ↓
Dense: 512
  ↓
Upsampling
  ↓
Convolution
  ↓
Synthetic Image
```

ReLU can be used in intermediate generator layers, with a suitable output activation based on image normalization.

Discriminator

The discriminator receives both real and generated images.

$$D(X) \rightarrow P(real)$$

It attempts to distinguish real images from synthetic images.

Example:

```
Image
  ↓
Convolution
  ↓
Leaky ReLU
  ↓
Convolution
  ↓
Leaky ReLU
```

↓
Dense
↓
Sigmoid
↓
Real / Fake

Adversarial Training

The generator and discriminator compete with each other.

Discriminator

The discriminator tries to:

- Classify real images as real
- Classify generated images as fake

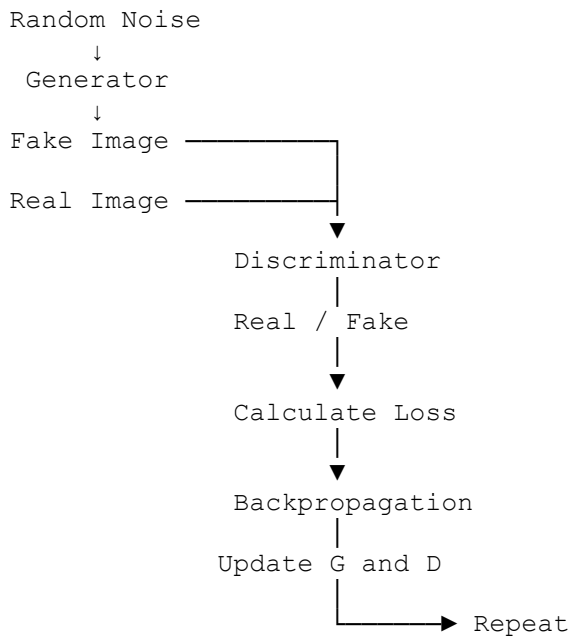
Generator

The generator tries to produce images that the discriminator classifies as real.

The original GAN objective is:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$$

Training Workflow



Evaluation

Generated images can be evaluated using:

- Visual quality
- Diversity of generated samples
- FID (Fréchet Inception Distance)
- Inception Score
- Similarity to the real-data distribution
- Downstream classifier performance

GAN vs Autoencoder

Feature	Autoencoder	GAN
Main purpose	Representation/reconstruction	Data generation
Structure	Encoder + Decoder	Generator + Discriminator
Latent space	Explicit latent representation	Random latent vector
Training	Reconstruction loss	Adversarial loss
Generated samples	Can be blurry	Usually more realistic/sharp
Best use	Feature learning/compression	Synthetic data generation

Why GAN is Suitable

The problem specifically requires **realistic synthetic images** from limited training data. GANs directly optimize a generator against a discriminator, encouraging generated samples to resemble the real data distribution.

Expected Outcome

The GAN generates diverse synthetic images that can supplement the limited real training dataset and potentially improve the performance of downstream image-processing or classification systems.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.

Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, wellorganized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.